1. Longest substring without repeating characters. (Optimal O(n) sliding window)

Question: Given a string s, find the length of the longest substring (a continuous block of characters) that does not contain any repeating characters. The solution should be implemented using the optimal $O(n)$ **Sliding Window** technique.

Simple Explanation: We use a **Sliding Window** with a **HashMap** to solve this in a single pass. The map stores the last seen index of each character. We expand the window (right pointer). If we find a repeating character, we immediately move the window's left pointer past the previously seen index of that character. We update the maximum length found at every step.

```
import java.util.HashMap;
import java.util.Map;

public class LongestUniqueSubstring {
    /**
     * @param s The input string.
     * @return The length of the longest substring without repeating
characters.
     */
    public static int lengthOfLongestSubstring(String s) {
        if (s == null || s.isEmpty()) {
            return 0;
        }

        // Map to store character -> index of its last occurrence
        Map<Character, Integer> charMap = new HashMap<>();
        int left = 0; // Left pointer of the window
        int maxLength = 0;

        // Right pointer expands the window
        for (int right = 0; right < s.length(); right++) {
            char currentChar = s.charAt(right);

            // If the character is repeating AND is inside the current
window boundary
            if            (charMap.containsKey(currentChar)            &&
charMap.get(currentChar) >= left) {
                // Move the left pointer past the last seen index of the
repeating character
                left = charMap.get(currentChar) + 1;
            }

            // Update the map with the current character's new index
            charMap.put(currentChar, right);

            // Update the maximum length found: current window size is
(right - left + 1)
            maxLength = Math.max(maxLength, right - left + 1);
        }
```

```
        return maxLength;
    }
}
```

2. Find all permutations of a string (backtracking).

Question: Write a function that finds and prints all unique permutations (arrangements) of the characters in a given string using a **Backtracking** algorithm.

Simple Explanation: Permutation uses a **Backtracking** approach. We fix one character at the start, and then recursively find all permutations for the remaining characters. After the recursive call returns, we "undo" the choice (backtrack) and try the next character in the starting position. This guarantees every possible arrangement is generated exactly once.

```
import java.util.ArrayList;
import java.util.List;

public class StringPermutations {
    /**
     * Finds and prints all unique permutations of a string.
     */
    public static List<String> findPermutations(String str) {
        List<String> results = new ArrayList<>();
        if (str != null) {
            permute(str.toCharArray(), 0, results);
        }
        return results;
    }

    private static void permute(char[] chars, int start, List<String>
results) {
        // Base case: If start pointer reaches the end, a full permutation
is found
        if (start == chars.length - 1) {
            results.add(new String(chars));
            return;
        }

        // Recursive step: Try placing every character at the current
'start' position
        for (int i = start; i < chars.length; i++) {
            // 1. Choose: Swap the character at 'i' with the character at
'start'
            swap(chars, start, i);

            // 2. Explore: Recurse to find permutations for the rest of
the string
            permute(chars, start + 1, results);
```

```
            // 3. Unchoose (Backtrack): Swap them back to restore the
original order
            swap(chars, start, i);
        }
    }

    private static void swap(char[] chars, int i, int j) {
        char temp = chars[i];
        chars[i] = chars[j];
        chars[j] = temp;
    }
}
```

3. Implement basic string compression/decompression.

Question: Implement a basic string compression function (e.g., aabccc -> a2b1c3) and a corresponding decompression function. The compression should only be applied if it actually reduces the string length.

Simple Explanation: For **compression**, we loop through the string, counting consecutive identical characters. When the character changes, we append the character and its count. For **decompression**, we loop through the compressed string, repeating each character by the number indicated immediately following it.

```
public class BasicCompressor {

    /** Compression: aabccc -> a2b1c3 */
    public static String compress(String str) {
        if (str == null || str.isEmpty()) {
            return str;
        }

        StringBuilder compressed = new StringBuilder();
        int count = 1;

        for (int i = 0; i < str.length(); i++) {
            // Count consecutive identical characters
            if (i + 1 < str.length() && str.charAt(i) == str.charAt(i +
1)) {
                count++;
            } else {
                // Append the character and its count
                compressed.append(str.charAt(i));
                compressed.append(count);
                count = 1; // Reset count
            }
        }
        // Only return the compressed string if it's actually shorter
        return compressed.length() < str.length() ? compressed.toString()
: str;
    }
```

```
    /** Decompression: a2b1c3 -> aabccc */
    public static String decompress(String compressed) {
        if (compressed == null || compressed.isEmpty()) {
            return compressed;
        }

        StringBuilder decompressed = new StringBuilder();

        for (int i = 0; i < compressed.length(); i++) {
            char character = compressed.charAt(i);

            // Check if the next element is a digit (the count)
            if     (i     +     1     <     compressed.length()     &&
Character.isDigit(compressed.charAt(i + 1))) {

                //  Read  the  count  (assuming  single  digit  counts  for
simplicity)
                int count = Character.getNumericValue(compressed.charAt(i
+ 1));

                // Append the character 'count' times
                for (int j = 0; j < count; j++) {
                    decompressed.append(character);
                }
                i++; // Skip the count digit in the main loop
            } else {
                 // Invalid format or character at the end (just append
the character)
                 decompressed.append(character);
            }
        }
        return decompressed.toString();
    }
}
```

4. Check if two strings are isomorphic.

Question: Determine if two strings, s and t, are isomorphic. Two strings are isomorphic if the characters in s can be replaced to get t, maintaining the same mapping for every character (e.g., egg and add are isomorphic because e maps to a and g maps to d).

Simple Explanation: We use **two HashMaps** to track the mapping of characters from s to t and from t back to s simultaneously. We iterate through both strings together; if we find a character that already has a mapping but the mapping is incorrect, or if a character in t is already mapped to a different character in s, the strings are not isomorphic.

```
import java.util.HashMap;
import java.util.Map;

public class IsomorphicChecker {
```

```
    /**
     * @param s The first string.
     * @param t The second string.
     * @return true if s and t are isomorphic, false otherwise.
     */
    public static boolean areIsomorphic(String s, String t) {
        if (s == null || t == null || s.length() != t.length()) {
            return false;
        }

        // Map 1: s -> t mapping
        Map<Character, Character> sToT = new HashMap<>();

        // Map 2: t -> s mapping (to ensure one-to-one mapping)
        Map<Character, Character> tToS = new HashMap<>();

        for (int i = 0; i < s.length(); i++) {
            char charS = s.charAt(i);
            char charT = t.charAt(i);

            // Check consistency for s -> t
            if (sToT.containsKey(charS)) {
                // If mapping exists, it must map to the same charT
                if (sToT.get(charS) != charT) {
                    return false;
                }
            } else {
                // Check consistency for t -> s (No two characters in s
can map to the same character in t)
                if (tToS.containsKey(charT)) {
                    return false;
                }

                // Establish new, consistent mapping
                sToT.put(charS, charT);
                tToS.put(charT, charS);
            }
        }
        return true;
    }
}
```